
MANTHAN

AI-Powered Research & Innovation Copilot

Detailed Project Document

Project Members

Sr. No.	Name	Email
1.	Sahil Memane	sahilmemane007@gmail.com
2.	Sagar Kharat	sagarkharat138@gmail.com
3.	Shivam Bande	shivambande18@gmail.com
4.	Shreya Patil	shreyapatil9681@gmail.com
5.	Rutuja Khadse	rutujakhadse2006@gmail.com

Table of Contents

Table of Contents	2
1. Executive Summary & Core Value Proposition	4
1.1 Executive Summary	4
1.2 System Vision, Scope & Primary Objectives	4
1.3 Key Differentiators & Industry Alignment	4
2. System Architecture & Technical Design	6
2.1 Overall System Architecture & Data Flow	6
2.2 Comprehensive Technology Stack	7
2.3 Modular System Components	8
2.4 Infrastructure & Scalability Strategy	8
3. Repository & Directory Structure	9
3.1 Folder Hierarchy & File Layout	9
3.2 Key File Responsibilities	10
4. Detailed Feature Breakdown & User Workflows	11
4.1 Security & Credential Management	11
4.2 Core Business Logic & Execution Engines	11
4.2.1 Agent 1 — Idea Intake & Validation	12
4.2.2 Agent 2 — DeepSearch: Multi-Source Research	13
4.2.3 Agent 3 — Existing Solution Analysis & Gap Detection	13
4.2.4 Agent 4 — Tech Stack & Architecture Recommendation	14
4.2.5 Agent 5 — Resource Recommendation	145
4.2.6 Agent 6 — Roadmap & Milestone Planning	145
4.2.7 Agent 7 — Risk, Feasibility & Compliance	156
4.2.8 Agent 8 — Documentation Synthesis (Master Document Assembly)	166
4.3 User Interface, Dashboards & Visualizations	166
4.4 Real-Time Systems & Integration Layers	166
5. Mathematical Foundations & Quantitative Frameworks	17
5.1 Core Mathematical Formulations & Algorithms	178
5.1.1 Idea Validation Score	178
5.1.2 Resource Relevance Score (Agent 5)	178
5.1.3 Milestone Duration Estimation (PERT-style Three-Point Estimate)	189
5.1.4 Freshness / Activity Decay Function	189
5.2 Statistical Models & Scheduling Simulations	189
5.2.1 Critical Path Method (CPM) for Roadmap Sequencing	189
5.2.2 Downstream-Impact Propagation Score	20
5.3 Analytical Calculators & Infrastructure Indicator Derivations	20
5.3.1 Source Credibility Aggregation	20
5.3.2 Token-Bucket Rate Limiting (Gateway)	20
5.3.3 Concurrency & Queueing — Little's Law for Agent Worker Sizing	20

5.3.4 Cache Effectiveness	20
5.4 Mathematical Formulation Summary Matrix	21
6. Database Schema & Security Infrastructure	22
6.1 Entity-Relationship Overview	22
6.2 Collection Schema Definitions & Migration Scripts	22
6.2.1 users	22
6.2.2 projects	22
6.2.3 project_intake	23
6.2.4 agent_runs	23
6.2.5 refinement_threads, master_documents, recommended_resources, sources	23
6.3 Access Control Policies (Row-Level-Security Equivalent)	23
7. Local Development & Installation Guide	24
7.1 Prerequisites & System Requirements	24
7.2 Step-by-Step Setup & Configuration	25
7.3 Environment Variables Reference	26
7.4 Running the Local Development Suite	27
8. Verification, Testing & Quality Assurance	28
8.1 Static Type Analysis & Code Audits	28
8.2 Algorithm & Agent-Output Validation	29
8.3 Operational Guardrails & Edge-Case Handling	29
9. Multi-Platform Deployment & DevOps Pipeline	30
9.1 Static Web Deployment (Vercel)	30
9.2 Containerized Deployment (Docker / Cloud Run)	30
9.3 Native Mobile & Extension Compilation	31
Android (React Native / Expo)	31
Chrome Extension (Manifest V3)	31
10. Project Changelog, Roadmap & License	32
10.1 Operational History & Version Commit Log	32
10.2 Future Roadmap & Milestone Objectives	32
10.3 Troubleshooting Matrix	33
10.4 Error Log Book	33
10.5 Licensing & Intellectual Property Terms	34

1. Executive Summary & Core Value Proposition

1.1 Executive Summary

Manthan is an agentic AI project architect that takes one structured idea submission, and generates a completely validated, researched, architected and export ready Master Project Document. It is designed for students, first time builders, organizers of hackathons and faculty mentors who spend hours now searching for relevant information across search engines, Github, Kaggle, and YouTube only to manually curate what they find into a neat, submission-ready document.

Manthan is not a single question that can be posed to a language model on project planning. Rather, it breaks the problem down into eight distinct special agent processes: idea validation, multi-source research, gap identification, architecture recommendations, resource curation, roadmap planning, risk assessment and documentation synthesis. When each agent completes its reasoning, it returns a structured result with a citation to the student, using its own bounded reasoning loop, tools and memory. Most importantly, each agent's output may be separately edited using a “refine this” dialogue, which does not affect other layers—a feature that directly tackles the biggest usability flaw of the single-prompt planning tools – discarding an entire plan after one of its elements is incorrect.

If all layers are accepted (or the default layers that are owned by the agent), a dedicated Documentation Synthesis Agent combines all of the seven layers into one single Document, which is cross-referenced and rich with diagrams, and renders each diagram into an SVG file, styles the whole document in an HTML template, and creates a paginated, print-ready PDF – the Master Project Document. It's the same document that you can access from a full web app, native Android app, or Chrome extension for capturing ideas while you're in the moment.

1.2 System Vision, Scope & Primary Objectives

Manthan wants to bring the gap between a “I have an idea” and a “ I have a build-ready, professionally documented project plan” down to the nitty gritty and make it as reliable as possible without keeping the reason behind the creation of the plan from that person who must explain it to a jury or mentor. The system scope includes structured intake, multi-agent orchestration, human-in-the-loop refinement and cross-platform document delivery.

1.3 Key Differentiators & Industry Alignment

Most current AI planning tools, as well as those that generate plans based on an idea (“idea-to-plan”), are overwhelmingly defined by a single large-context generation call: the student types in an idea, and the AI generates the answer as one long text document. This is a pattern that bundles together eight distinct kinds of reasoning skills: validate an idea, rank search results, diff against prior art, select a stack, sequence milestones, evaluate risk, and write prose into a single opaque generation. If one part of it is incorrect, then fixing that part requires a re-creation of all the parts and loss of what is correct. It's not about the looks, it's about the architecture: Manthan's differentiation.

Dimension	Typical Single-Prompt Tool	Manthan
Correction granularity	Regenerate the entire plan	Refine exactly one agent layer
Constraint handling	Stack/budget treated as soft hints	Structured intake is a hard contract per agent
Traceability	Unsourced or loosely cited claims	Every claim resolves to a URL; unsupported claims are dropped
Prior-art awareness	Often absent or superficial	Dedicated gap-detection agent runs before architecture
Output artifact	Chat transcript or single webpage	Versioned, paginated, submission-ready PDF
Platform reach	Single web surface	Web, Android, and Chrome extension on one shared session

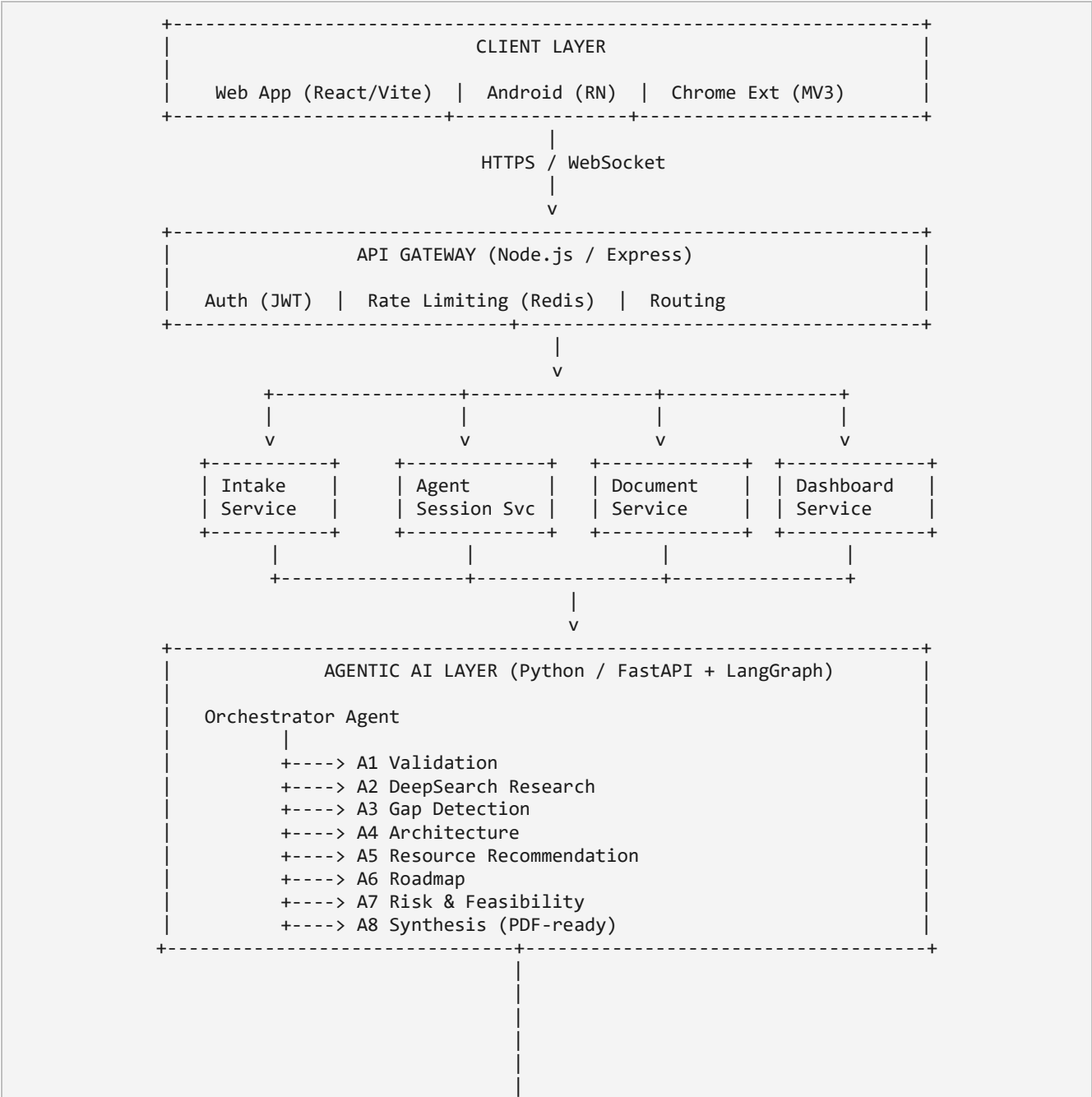
This places Manthan in alignment with the general trend in the industry from single-shot generative assistants to multi-agent, tool-augmented systems that have explicit, well-defined state and human-in-the-loop checkpoints — as opposed to the randomise prompt chaining of single-shot systems.

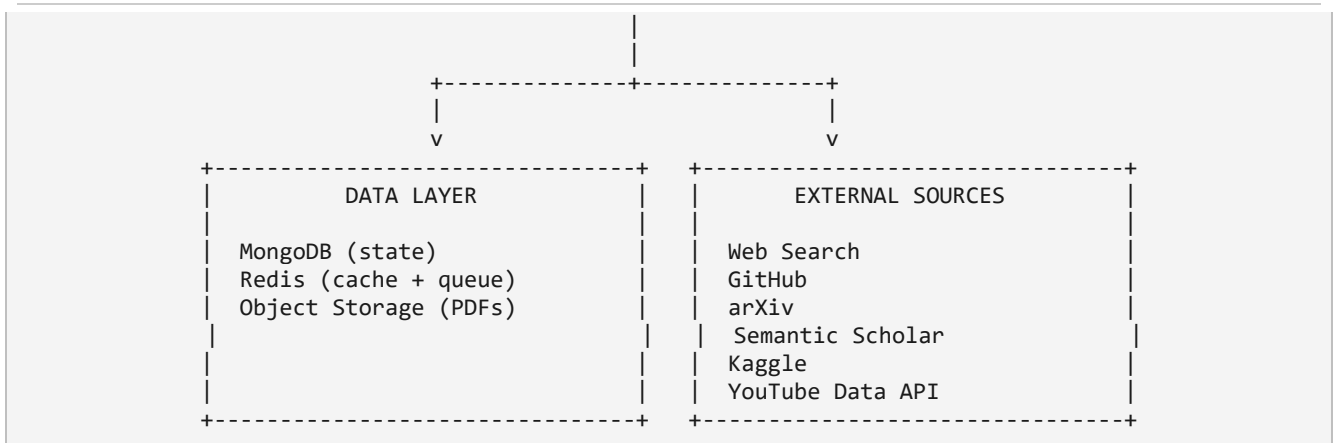
2. System Architecture & Technical Design

2.1 Overall System Architecture & Data Flow

There are five layers in Manthan: Client Layer (web, Android, Chrome extension), API Gateway Layer (authentication, rate limiting, routing), Core Services in Node.js (intake, agent session, document export, dashboard), Agentic AI Layer in Python/FastAPI (Orchestrator and eight layer agents), and Data Layer (MongoDB, Redis, object storage). These external research sources, such as a web search API, GitHub, arXiv/Semantic Scholar, Kaggle, and YouTube are only accessed by the Agentic AI Layer and not directly by the client or gateway.

Figure 2.1 — High-level component architecture (text representation)





All other artifacts (agent output, refinement, master document) are based on that unchanging record, plus whatever the agents pull at run time. The Orchestrator keeps all agents output as new immutable versions, never overwriting the old version, and the complete provenance of the final PDF is always retrievable.

2.2 Comprehensive Technology Stack

Technologies are grouped by concern below; the full stack with per-technologyz detailed in Section 10.

Layer	Primary Technologies	Rationale
Frontend (Web)	React, Vite, TypeScript, Tailwind CSS, Zustand, Socket.IO client	Fast dev loop, strict typing for all API contracts, and low overhead state management for an agent UI with a card-heavy board.
Mobile	React Native (Expo), React Navigation	Instead of having a second implementation, shares design tokens and API client with the web app
Browser Extension	Manifest V3, React (popup), Service Worker	Lightweight quick-capture surface, without the need to duplicate the complete agent board.
API Gateway & Services	Node.js, Express, Zod, BullMQ	Async has come a long way; Zod provides compile-time-close validation with requests; BullMQ separates the slow agent jobs from the HTTP request/response cycle.
Agentic AI Core	Python, FastAPI, LangGraph	The execution DAG (Section 2.3) and the refinement loop are explicitly modeled as graph state in LangGraph.
Database	MongoDB Atlas, Redis (Upstash)	A document model is more suitable for schemas with multiple different outputs from the agents to cache, than a fixed relational schema, and Redis supports this queue. The BullMQ queue is backed by Redis, which is suitable for document model as its schemas are heterogeneous, meaning that multiple different outputs from the agents can be placed in the same document.
Document Rendering	Mermaid CLI, Playwright (headless Chromium)	Pre-rendered diagrams in SVG, so there's no runtime js dependency when generating PDFs, paginated & print-accurate PDFs from headless Chromium.
Cloud / Hosting	Vercel, Google Cloud Run, Cloudflare R2	Container native hosting for all backend/AI services that is free tier friendly; zero egress object storage for pdfs.

2.3 Modular System Components

The system is designed to be composed of small, independently deployable services – like the “independent, regenerable layer” approach within the agent DAG itself:

2.4 Infrastructure & Scalability Strategy

Each backend and AI service is delivered as a standalone container, and Google Cloud Run scales in size horizontally since there is no "in memory" session state: session and job state is stored in MongoDB and Redis, not in a process.

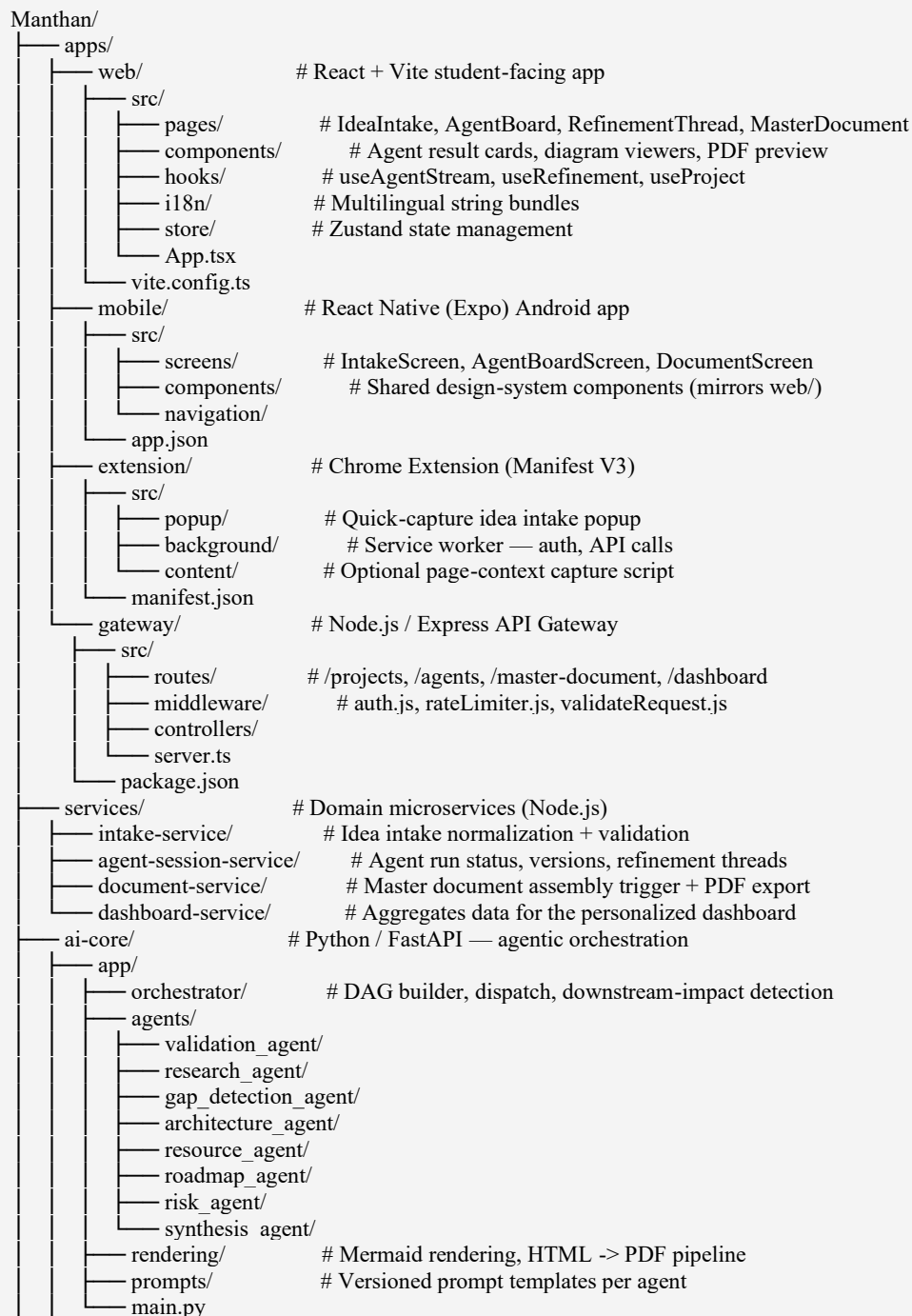
Long-running or externally-throttled work (agent invocations, PDF rendering) is detached from the request/response cycle; if the work takes a long time to run, it will be put in a BullMQ background job, not done synchronously within the HTTP request.

3. Repository & Directory Structure

3.1 Folder Hierarchy & File Layout

Manthan is a monorepo consisting of client applications (apps/), Node.js domain microservices (services/), the Python agentic core (ai-core/), infrastructure definitions (infra/), and cross-cutting documentation/tests. This design ensures that each client surface is using the same Gateway contract, and that the agent implementations are not dependent on the services that only call them.

Figure 3.1 — Full repository tree



```

├── requirements.txt
├── infra/
│   ├── docker/
│   ├── cloud-run/
│   └── docker-compose.yml
├── docs/
│   ├── architecture/
│   ├── api/
│   └── README.md
├── scripts/
│   ├── seed-db.ts
│   └── generate-openapi.ts
├── tests/
│   ├── unit/
│   ├── integration/
│   └── e2e/
├── .github/
│   └── workflows/
├── .env.example
└── README.md

```

3.2 Key File Responsibilities

Path	Responsibility
apps/web/src/hooks/useAgentStream.ts	Opens and manages the WebSocket session for completing events for each Agent to the Agent Board UI
apps/gateway/src/middleware/rateLimiter.js	A Redis-backed sliding window limiter that has more stringent limits for refining endpoints than read endpoints.
services/intake-service/normalizer.ts	Uses the Intake Normalizer (Section 4.1.3): removes/cleans text or tags the domain when it is under-length
services/agent-session-service/versioning.ts	Re-writes new agent_runs versions in place and never mutates a previous version in place.
ai-core/app/orchestrator/dag_builder.py	Reads the validated input, and builds the execution DAG as outlined in Section 4.2 (Agentic System Overview).
ai-core/app/orchestrator/impact_detector.py	Whether or not a refinement would render the input snapshot of a downstream agent invalid, and whether the agent needs it to be reviewed
ai-core/app/agents/*/pipeline.py	The Internal Reasoning Pipeline given to that agent (as defined per-agent in Section 4.2)
ai-core/app/rendering/mermaid_render.py	Before HTML composition, pre-renders all the Mermaid diagrams created from Agent 4 and Agent 8 into SVG.
ai-core/app/rendering/pdf_render.py	Starts the HTML -> PDF render step of Section 6.3, using the headless-Chromium.
infra/docker/*.Dockerfile	A single Dockerfile for each service, each service can be built and deployed independently of the others to Cloud Run.
scripts/generate-openapi.ts	Generates the OpenAPI specification which is used by the API Design part (Feature breakdown / integration layer, Section 4).

4. Detailed Feature Breakdown & User Workflows

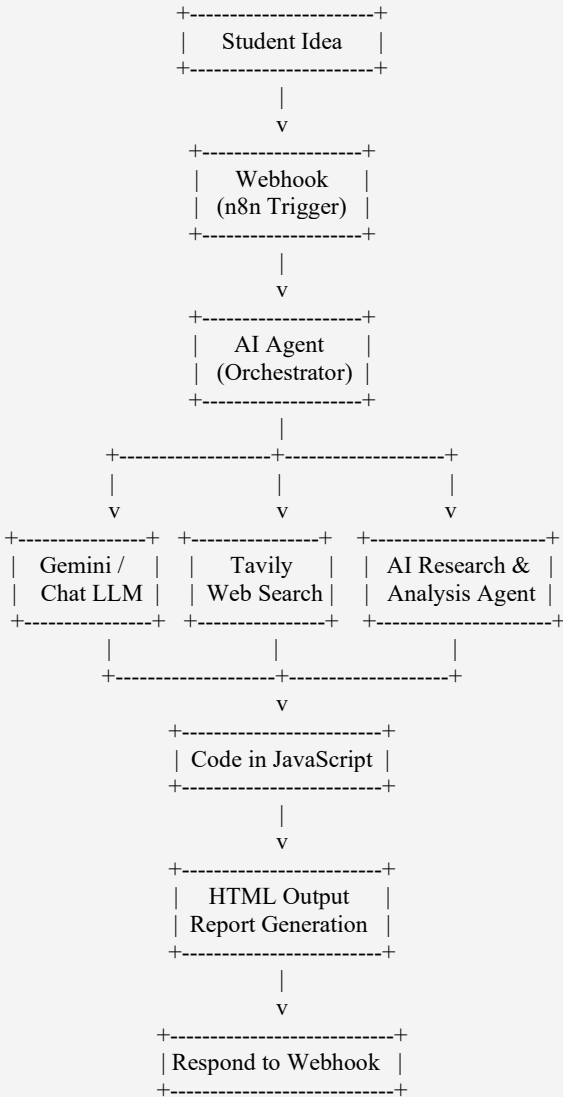
4.1 Security & Credential Management

The design of Manthan's credential management requires that all three client surfaces (Web, Android and Chrome extension) share one account, and then have a set of agents that call external, cost-metered third-party APIs on the student's behalf. These facts lead to the design below.

4.2 Core Business Logic & Execution Engines

The Orchestrator Agent manages eight special agents where the core business logic of Manthan resides. Each agent is documented below with the same structure of fixed format, to keep layers directly comparable. All eight agents are treated the same by the Orchestrator and UI because they share a common lifecycle (Queued → Running → AwaitingReview ↔ Refining → Accepted) with a bounded automatic retry if the tool fails (unrecoverably).

Agent Architecture Workflow Prototype (using n8n):



4.2.1 Agent 1 — Idea Intake & Validation

Decides if the idea is coherent, is of an appropriate size, is original and worthy of research and makes a reasoned judgement rather than a pass/fail judgement.

IP Internal Reasoning Pipeline: (1) Intent and Domain classification; (2) Coherence check – does proposed solution actually solve the problem?; (3) Quick pass for originality (low depth, not agent 2's depth); (4) Verdict synthesis – return score and originality flag and reasoned scope-adjustment suggestions.

Attribute	Detail
Trigger & Position	The first agent to run, after the agents have been normalized on intake; doesn't prevent any agent downstream of it from running, but its verdict is shown along with every agent that follows in subsequent layers to provide context.
Inputs	Idea Statement, Initial Flow, Problem Being Solved, Proposed Solution, Target Users, Domain tags, Non-Negotiables.
Tools & Function Calls	Lightweight web search (low result-count fast-path), a domain classifier and a structured-output LLM call.
Output Schema	It has a validation score ranging from 0 to 100, an originality flag, detected domain, scope verdict, scope adjustment suggestions, and reasoning notes.
Customization Interface	Student re-prompts with corrections; the agent re-scopes its verdict without re-running research, e.g. this is a 2-week capstone rather than a hackathon project.
Handoff	Targeting Agent 2 and Roadmap sizing Agent 6 are picking up domain and scope verdict information.

4.2.2 Agent 2 — DeepSearch: Multi-Source Research

Automatically constructs a ranked collection of research resources from the web, code repositories, academic papers, and data sets—replacing several hours of tedious multi-tab searches.

Attribute	Detail
Trigger & Position	Runs immediately after Agent 1 generates a domain tag and doesn't have to wait for Agent 1's verdict as targeting only requires the domain.
Inputs	Idea Statement, detected domain, Known References/Inspirations, optional filters for source-type.
Tools & Function Calls	A Web search API, GitHub API, arXiv/Semantic Scholar API, Kaggle API, and YouTube Data API for fetching information, headless-browser fetch for non-API sources, and an embedding-based re-ranker.

Attribute	Detail
Output Schema	This is a summary of the research findings, along with other information such as type, title, url, author, published_date, and credibility_score. This is a summary of the research findings, as well as other information, including type, title, url, author, published_date and credibility_score.
Customization Interface	A scope change from student (narrowing) or wider (broadening) scope; scope change is re-run (fan-out) without re-researching from scratch.
Handoff	Supplies an entire ranked source to Agent 3 (gap detection) and Agent 5 (recommender agent).

Internal Reasoning Pipeline: (1) query expansion into 3-5 targeted queries; (2) parallel fan-out to web search, GitHub, arXiv/Semantic Scholar, Kaggle and YouTube; (3) deduplication of near-duplicate results; (4) credibility and recency ranking by incorporating source-type priors and repository activity signals; and (5) citation-enforced summarization, discarding claims that cannot be grounded on retrieved sources and not showing them unsupported. Builds a ranked, citation-backed research corpus spanning the web, code repositories, academic papers, and datasets, replacing hours of manual multi-tab searching.

4.2.3 Agent 3 — Existing Solution Analysis & Gap Detection

Existing Solution Analysis & Gap Detection

Helps students avoid the danger of re-inventing a solved problem by presenting them with an actionable answer to “what already exists and where is the real gap?” to the pile of research results.

Internal Reasoning Pipeline: (1) build a thematic cluster of the corpus by the (solution) approach; (2) compare and contrast the idea with each cluster, and classify features as solved, partially-solved, or an open gap; (3) differentiate and synthesize the idea by ranking gaps by defensibility/busability based on team size/duration/budget; (4) write out a side-by-side comparison matrix with closest 3-5 existing solutions.

Attribute	Detail
Trigger & Position	Acts in parallel with Agent 5, after Agent 2 is delivering a research corpus.
Inputs	Research Corpus (Agent 2), Idea Statement, Proposed Solution, Known References/Inspirations.
Tools & Function Calls	Comparative reasoning with LLM over the research corpus with a dedicated vector database at this scale of corpus. Comparative reasoning with LLM over the research corpus (no dedicated vector database at this corpus size).
Output Schema	The solution_clusters are the same as in score_input.solution_clusters (as in score_input) and comparison_matrix.gap_categories (already_solved, partially_solved, open_gaps) and differentiation_ranking ().
Customization Interface	Student re-prompts, saying, “Weigh technical novelty over market novelty” or “only consider the offline-use-case angle in the gap analysis.”
Handoff	Provides gap categories and differentiation ranking to Agent 4 (architecture should be oriented towards the real gap) and Agent 6 (roadmap should focus on early proof of the gap).

4.2.4 Agent 4 — Tech Stack & Architecture Recommendation

Creates a technical architecture and stack that comes from a specific solution; that is, doesn't take any student provided stack as a recommendation to override.

Internal Reasoning Pipeline: (1) constraint extraction – separating hard constraints from open decisions; (2) component decomposition of mental-model flow to client, API layer, data store, AI/ML component and external integrations; (3) selection of the stack per open component with explicit justification against at least any rejected alternative; (4) generation of the architecture diagram as a Mermaid source; (5) explicit documentation of the trade-offs between what was sacrificed for what.

Attribute	Detail
Trigger & Position	For both Agent 1 (validation/scope) and Agent 3 (gap analysis), there are both options.
Inputs	Idea Statement, Initial Flow, Suggested Tech Stack (if any), Detected domain, gap categories, Team Size & Duration, Budget, Non-Negotiables.
Tools & Function Calls	Structured output LLM call (JSON-schema enforced) and a Mermaid diagram generator.
Output Schema	architecture_summary, component_breakdown[], recommended_stack[] (component, technology, justification, alternative_considered), architecture_diagram_source, trade_off_notes[].
Customization Interface	The most common layers that are refined: “no Firebase”, “assume production scale, not hackathon scale”, “swap the ML component for a rule based one”. Affected components only change, the diagram is re-derived.
Handoff	Gets stack and component breakdown from the selected stack and feeds it to Agent 6 (roadmap sequencing) and Agent 7 (feasibility/risk).

4.2.5 Agent 5 — Resource Recommendation

Exposes relevant datasets, repositories, papers, and tutorials for the actual construction of the idea, rather than general “learn X” links.

Internal Reasoning Pipeline: (1) resource-type triage to datasets, repositories, papers, and learning material, (2) relevance scoring based on the idea, rather than general popularity of a domain, (3) freshness and activity checks (commit recency, star trajectory, dataset-lastupdated-date), (4) stack-aware filtering that deprioritizes resources in an incompatible ecosystem.

Attribute	Detail
Trigger & Position	Runs in parallel with Agent 3, after the research corpus of Agent 2 is available.
Inputs	An agent might be needed to detect the domain of the research corpus and/or recommend the tech stack (if any) for the research.
Tools & Function Calls	The GitHub API, Kaggle API, arXiv/Semantic Scholar API, and the YouTube Data API.
Output Schema	The types of datasets, repositories, papers, and tutorials.
Customization Interface	Students re-state that they want only free/open-license datasets or that they will prioritize repos that they can “fork and extend” in a week.
Handoff	Does not depend on any other analytical agent, but is fed by the Synthesis Agent's Resources section.

4.2.6 Agent 6— Roadmap & Milestone Planning

This module teaches students how to create road maps and milestones for agents and the planning process. Maps the validated concept, gap analysis and architecture into a tangible sequence of milestones, bounded in time.

Internal Reasoning Pipeline: (1) Milestones extracted from component breakdown, ordered by the build dependency; (2) Gap-first sequencing (the component actually differencing the idea is built first, not last); (3) Duration estimation based on team size and total duration, in order to highlight any unrealistic fit; (4) Critical-path identification (identifying which milestones are blocking other milestones).

Attribute	Detail
Trigger & Position	After Agent 4 (stack) and Agent 6 (roadmap) are available, runs last.
Inputs	Architecture and stack, milestone list and fit warnings, Non-Negotiables, domain.
Tools & Function Calls	Call to LLM reasoning on the output of Agents 4 & 6.
Output Schema	The <code>technical_risks[]</code> and <code>scope_risks[]</code> are a pair of new variables that carry a risk and a mitigation. The new variables <code>technical_risks[]</code> and <code>scope_risks[]</code> both contain a risk and a mitigation.
Customization Interface	Student re-prompts “don't flag the learning curve” or “we already have institutional data-handling approval.”
Handoff	Fills in directly into the Synthesis Agent's Risk & Feasibility section.

4.2.7 Agent 7 — Risk, Feasibility & Compliance

Identifies what might go wrong or prevent completion on a surface — technical, scope, and data/compliance risk — before student begins construction.

Internal Reasoning Pipeline: (1) Technical Risk Scan – steep learning curves or components that have no fall-back; (2) Scope/Feasibility Cross-check – compliance and/or data-sensitive domain (framed in terms of “things to ask a mentor about”, NOT as legal advice); (3) Mitigation Drafting – at least one concrete mitigation for each flagged risk.

Attribute	Detail
Trigger & Position	Gated behind all seven analytical agents (Accepted or explicitly-defaulted); can be manually gated early for partial export of draft.
Inputs	Final output of all of the agents (1-7) and original intake record.
Tools & Function Calls	This include long-context structured-output LLM call, Mermaid renderer, and citation-deduplication logic.
Output Schema	The following elements should not be included in the generated document: <code>master_document_markdown</code> , <code>diagrams[]</code> , <code>bibliography[]</code> , <code>generation_metadata</code> (<code>agent_versions_used</code> , <code>generated_at</code>).
Customization Interface	Student re-prompts at the document-wide level for example, “make the executive summary shorter,” “reorder risk before roadmap” without re-running any of the analytical agents.
Handoff	The Output is directly fed into the PDF Export Pipeline (Section 9 of this document).

4.2.8 Agent 8 — Documentation Synthesis (Master Document Assembly)

Gathers all accepted agents to one single document that is cross-referenced, well-formatted and professional. Passes this document to the PDF export pipeline.

Internal Reasoning Pipeline: (1) Cross Reference Resolution: ensure that terminology and component names are consistent between sections written by different agents; (2) Narrative Assembly: compile the prose (executive summary etc) into a consistent document structure that appears as one coherent document; (3) Diagram Consolidation: ensure that diagrams use a consistent style; (4) Citation-appendix Compilation: ensure that the document's bibliography is a single deduplicated bibliography; (5) Document Structuring: compile the document into a fixed master-document template as defined in Section 6.2 of this document's framework.

Attribute	Detail
Trigger & Position	Gated behind all seven analytical agents reaching Accepted or explicitly-defaulted state; can be manually triggered early for a partial draft export.
Inputs	The final accepted output of every agent (1–7), plus the original intake record.
Tools & Function Calls	Long-context structured-output LLM call, Mermaid renderer, citation-deduplication logic.
Output Schema	master_document_markdown, diagrams[], bibliography[], generation_metadata (agent_versions_used, generated_at).
Customization Interface	Student re-prompts at the document-wide level (“make the executive summary shorter,” “reorder risk before roadmap”) without re-running any analytical agent.
Handoff	Output feeds directly into the PDF Export Pipeline (Section 9 of this document).

4.3 User Interface, Dashboards & Visualizations

Primary UI surface is the Agent Board: a card (validation, research, gap analysis, architecture, resources, roadmap, risk, synthesis) for each agent, each card's structured output rendered as soon as it streams in over WebSocket, regardless of whether any other cards are still running. Each card reveals three consistent affordances to view the current output, to open up the source/reasoning trail, and to display an inline “Refine this” prompt box (Section 4.4).

4.4 Real-Time Systems & Integration Layers

Manthan is designed around streaming, as agents execute as independent background jobs, and not polling. Immediately after sending a project to the Orchestrator, the Gateway returns. After sending a project to the Orchestrator, the Gateway returns immediately, via WebSocket (Socket.IO) is scoped to that project and get a separate event whenever an agent changes state – running to awaiting_review or from the new version when a refinement has been completed.

5. Mathematical Foundations & Quantitative Frameworks

Manthan is not a numerical-simulation product, so this section does not force artificial financial or physical models onto the system. Instead, it formalizes the quantitative machinery that genuinely governs the platform's behavior: the scoring functions each agent emits, the scheduling and duration mathematics behind the Roadmap Agent's critical path, the ranking and decay functions behind the Resource Agent, and the queueing, rate-limiting, and load models that govern how the orchestration layer behaves under concurrent traffic. Every formula below is drawn directly from a documented output field (`validation_score`, `relevance_score`, `credibility_score`, `critical_path`, `fit_warning`) or a documented infrastructure control (per-agent concurrency ceilings, sliding-window rate limits, cache TTLs).

5.1 Core Mathematical Formulations & Algorithms

5.1.1 Idea Validation Score

Agent 1 (Idea Intake & Validation) emits a `validation_score` in the range 0–100. It is computed as a weighted linear combination of three normalized sub-scores produced during the Internal Reasoning Pipeline: a scope-fit score, a coherence score, and an originality score.

$$V = 100 \cdot (w_1 S_{\text{scope}} + w_2 S_{\text{coh}} + w_3 S_{\text{orig}}), \quad w_1 + w_2 + w_3 = 1, \quad S_{\text{scope}}, S_{\text{coh}}, S_{\text{orig}} \in [0, 1]$$

LaTeX: $V = 100 \cdot (w_1 S_{\text{scope}} + w_2 S_{\text{coh}} + w_3 S_{\text{orig}}), \quad w_1 + w_2 + w_3 = 1$

Default weighting is $w_1 = 0.3$ (scope fit against team size / duration band), $w_2 = 0.45$ (coherence between problem and proposed solution), $w_3 = 0.25$ (originality relative to the lightweight prior-art pass). Coherence is weighted highest because a solution/problem mismatch is the failure mode Agent 1 is most explicitly designed to catch (Section 4.1).

- S_{scope} — 1.0 when the estimated build effort (Section 5.1.3) falls within the stated team-size/duration band; decays linearly as the estimate exceeds the band.
- S_{coh} — a normalized semantic-alignment score between the embedding-free LLM comparison of "Problem Being Solved" and "Proposed Solution / Approach."
- S_{orig} — 1 minus the maximum similarity score returned by the fast-path originality search against near-identical existing tools.

5.1.2 Resource Relevance Score (Agent 5)

Every candidate resource (dataset, repository, paper, tutorial) surfaced during Resource-Type Triage is scored for direct applicability rather than domain-general popularity, then filtered by stack compatibility.

$$R = \alpha \cdot M + \beta \cdot F + \gamma \cdot C, \quad \alpha + \beta + \gamma = 1$$

LaTeX: $R = \alpha M + \beta F + \gamma C, \quad \alpha + \beta + \gamma = 1$

Where M is a match score between the resource's extracted topic vector and the idea's domain/component vector (LLM-scored, 0–1), F is the freshness/activity term (Section 5.1.4), and C is a stack-compatibility indicator (1 if the resource's ecosystem matches the known or recommended stack, a configurable penalty value, default 0.4, otherwise). Default weights are $\alpha = 0.5$, $\beta = 0.2$, $\gamma = 0.3$.

5.1.3 Milestone Duration Estimation (PERT-style Three-Point Estimate)

Agent 6's Duration Estimation step assigns an estimated effort to each milestone derived from the component breakdown. Because a single point estimate is unreliable for a first-time builder, each milestone's duration is modeled with a three-point (optimistic / most-likely / pessimistic) estimate, following the classic PERT formulation:

$$E = (O + 4M + P) / 6, \quad \sigma = (P - O) / 6$$

LaTeX: $E = \frac{O + 4M + P}{6}, \quad \sigma = \frac{P - O}{6}$

O, M, and P (optimistic, most-likely, pessimistic hours) are produced per milestone by the LLM planning call, calibrated against the stated team size and stack familiarity implied by Agent 4's output. The `fit_warning` field is raised whenever the sum of expected durations along the critical path (Section 5.1.5) exceeds the available duration window by more than one standard deviation of the aggregated path variance.

$$\text{fit_warning} \Leftrightarrow \sum E_i (i \in \text{critical path}) > D_{\text{available}} + \sum \sigma_i^2 \text{ (aggregated over the path)}$$

LaTeX: $\text{fit_warning} \iff \sum_{i \in \text{CP}} E_i > D_{\text{available}} + \sqrt{\sum_{i \in \text{CP}} \sigma_i^2}$

5.1.4 Freshness / Activity Decay Function

For repositories and datasets, the Resource Agent discounts stale results using an exponential recency-decay term, so a highly-starred but abandoned repository is not over-ranked relative to an actively maintained one.

$$F = e^{(-\lambda \cdot \Delta t)}, \quad \Delta t = \text{days since last commit / update}$$

LaTeX: $F = e^{-\lambda \Delta t}, \quad \Delta t = \text{days since last activity}$

λ is tuned per resource type: $\lambda = 0.004$ for repositories (≈ 173 -day half-life) and $\lambda = 0.0015$ for datasets (≈ 462 -day half-life), reflecting that datasets remain useful far longer than actively-developed code without new commits.

5.2 Statistical Models & Scheduling Simulations

5.2.1 Critical Path Method (CPM) for Roadmap Sequencing

Agent 6's Critical-Path Identification step is a standard forward/backward-pass CPM calculation over the milestone dependency graph produced in Milestone Extraction.

- Forward pass — Earliest Start / Earliest Finish per milestone i : $ES(i) = \max(EF(j))$ for all $j \rightarrow i$; $EF(i) = ES(i) + E(i)$
- Backward pass — Latest Finish / Latest Start: $LF(i) = \min(LS(k))$ for all $i \rightarrow k$; $LS(i) = LF(i) - E(i)$
- Slack: $Slack(i) = LS(i) - ES(i)$. Milestones with $Slack(i) = 0$ constitute the `critical_path` array returned to the student.

$$\text{critical_path} = \{ i \in \text{Milestones} \mid Slack(i) = 0 \}$$

LaTeX: $\text{critical_path} = \{ i \in M \mid LS(i) - ES(i) = 0 \}$

5.2.2 Downstream-Impact Propagation Score

When a refinement changes an agent's output, the Orchestrator (Section 3.2, 5.2) must decide whether a downstream agent should be flagged `needs_review`. This is modeled as a dependency-weighted change-magnitude score against the downstream agent's stored `input_snapshot`:

$$\text{Impact}(d) = \sum \text{over } k \text{ in } \Delta(u) [w_k \cdot \text{match}(k, \text{input_snapshot}(d))]$$

LaTeX: $\text{Impact}(d) = \sum_{k \in \Delta(u)} w_k \cdot \text{match}(k, \text{input_snapshot}(d))$

$\Delta(u)$ is the set of changed fields in the upstream agent u 's new output; $\text{match}(k, \cdot)$ is a binary indicator (1 if field k is referenced by name/id inside the downstream agent's recorded input snapshot, else 0); w_k is a per-field materiality weight (e.g., a changed database technology in Agent 4's recommended_stack carries $w_k = 1.0$, while a changed justification string carries $w_k = 0.1$). needs_review triggers when $\text{Impact}(d)$ exceeds a threshold $\tau = 0.5$, calibrated so that cosmetic wording changes never cascade but structural changes (a swapped datastore, a removed milestone) always do.

5.3 Analytical Calculators & Infrastructure Indicator Derivations

5.3.1 Source Credibility Aggregation

Every citation persisted to the sources collection carries a `credibility_score` $\in [0,1]$. For domains with multiple corroborating sources (used in Agent 2's synthesis and Agent 3's comparison matrix), aggregate confidence in a claim uses a noisy-OR combination rather than a simple average, since independent corroboration should raise confidence super-linearly:

Conf(claim) = 1 - $\prod (1 - c_i)$, for each supporting source i with credibility c_i

LaTeX: $\text{Conf}(\text{claim}) = 1 - \prod_{i=1}^n (1 - c_i)$

5.3.2 Token-Bucket Rate Limiting (Gateway)

The API Gateway's per-user and per-IP limiters (Section 17) use a sliding-window token bucket. A request is admitted only while the bucket has capacity:

tokens(t) = min(B, tokens(t₀) + r · (t - t₀)); admit $\Leftrightarrow$ tokens(t) $\geq$ 1

LaTeX: $\text{tokens}(t) = \min(\text{big}(B, \text{tokens}(t_0) + r(t - t_0)), \text{quad } \text{admit} \iff \text{tokens}(t) \geq 1$

B is bucket capacity, r is the refill rate (requests/second). Refinement submissions use a separate, stricter bucket (lower B and r) than general API traffic, matching the "separate, stricter limit on refinement submissions per agent per project" control described in Section 17.

5.3.3 Concurrency & Queueing — Little's Law for Agent Worker Sizing

Per-agent-type worker concurrency ceilings (Section 19) are sized using Little's Law, relating the average number of in-flight jobs L to arrival rate λ and mean service time W :

$L = \lambda \cdot W$

LaTeX: $L = \lambda W$

For an agent type with an observed mean arrival rate λ (jobs/minute) and mean LLM+tool completion time W (minutes), the worker pool is provisioned so that the concurrency ceiling c satisfies $c \geq L$ at the 95th-percentile arrival rate, preventing a burst of research-heavy Agent 2 jobs from starving lightweight Agent 1 workers, as noted in Section 19.

5.3.4 Cache Effectiveness

Redis-cached research queries (Section 19) are evaluated with a standard hit-ratio and expected-cost-savings model:

HitRatio = $H / (H + M)$; Savings = HitRatio · $N \cdot C_q$

LaTeX: $\text{HitRatio} = \frac{H}{H+M}$, $\text{Savings} = \text{HitRatio} \cdot N \cdot C_q$

H and M are cache hits and misses over a measurement window, N is total query volume, and C_q is the average LLM/search-API cost per query — this directly informs the TTL tuning referenced in the caching strategy of Section 19.

5.4 Mathematical Formulation Summary Matrix

Metric	Governing Equation	Owning Agent / Component	Range
Validation Score (V)	$V = 100(w_1S_{scope} + w_2S_{coh} + w_3S_{orig})$	Agent 1 — Validation	0–100
Resource Relevance (R)	$R = \alpha M + \beta F + \gamma C$	Agent 5 — Resource	0–1
Milestone Duration (E, σ)	$E = (O + 4M + P) / 6, \sigma = (P - O) / 6$	Agent 6 — Roadmap	hours
Freshness Decay (F)	$F = e^{(-\lambda \Delta t)}$	Agent 5 — Resource	0–1
Critical Path / Slack	$Slack(i) = LS(i) - ES(i)$	Agent 6 — Roadmap	hours, 0 = critical
Downstream Impact	$Impact(d) = \sum w_k \cdot match(k, \cdot)$	Orchestrator Agent	0–1, $\tau = 0.5$
Claim Confidence	$Conf = 1 - \Pi(1 - c_i)$	Agent 2 / Agent 3	0–1
Rate Limiter Tokens	$tokens(t) = \min(B, tokens(t_0) + r \cdot \Delta t)$	API Gateway	0–B
Worker Concurrency (L)	$L = \lambda \cdot W$	Agent Session Service	jobs
Cache Hit Ratio	$HitRatio = H / (H + M)$	Redis Cache Layer	0–1

6. Database Schema & Security Infrastructure

This section covers two document-store equivalents to a traditional schema: JSON-Schema collection validators (the functional equivalents of SQL DDL constraint) and index-creation scripts (the functional equivalents of SQL migrations), in addition to the application-level access-control policies that replace the Row-Level Security defined in Postgres.

6.1 Entity-Relationship Overview

Eight collections model the complete life-cycle, from account to a document that is exported. The access control policies in Section 6.3 are based on a single-root ownership chain from users to projects: ownership of all users flows to all projects, and ownership of all downstream collections (intakes, agent runs, refinement threads, resources, sources, master documents) flows through the projects to users.

Collection	Purpose	Parent Scope
users	Student accounts and auth credentials	— (root)
projects	One record per idea/project a student starts	users
project_intake	Immutable Stage 1 structured intake submission	projects (1:1)
agent_runs	Every version of every agent's output, per layer	projects (1:N)
refinement_threads	Refinement conversation history per agent	projects (1:N)
master_documents	Each generated PDF version and agent versions used	projects (1:N)
recommended_resources	Datasets, repos, papers surfaced by Agent 5	projects (1:N)
sources	Individual citations linked to agent outputs	agent_runs (1:N)

6.2 Collection Schema Definitions & Migration Scripts

MongoDB doesn't have any native DDL, so schema constraints are enforced by application of \$jsonSchema collection validators through idempotent migration scripts using the official MongoDB driver. This gives the same guarantees SQL DDL offers: required fields, types and enumerations; and maintains the flexible, versioned-document model that the agent pipeline relies on.

6.2.1 users

```
// migrations/001_create_users.js
db.createCollection("users", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["name", "email", "preferred_language", "created_at", "updated_at"],
    properties: {
      name: { bsonType: "string" },
      email: { bsonType: "string", pattern: "^.+@.+\\.+$" },
      password_hash: { bsonType: ["string", "null"] },
      oauth_provider: { bsonType: ["string", "null"], enum: ["google", "none", null] },
      preferred_language: { bsonType: "string" },
      created_at: { bsonType: "date" },
      updated_at: { bsonType: "date" }
    }
  }
})
```

```

    }},
    validationLevel: "strict"
  });
  db.users.createIndex({ email: 1 }, { unique: true, name: "uniq_email" });

```

6.2.2 projects

```

// migrations/002_create_projects.js
db.createCollection("projects", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["user_id", "title", "status", "created_at", "updated_at"],
    properties: {
      user_id: { bsonType: "objectId" },
      title: { bsonType: "string", minLength: 1 },
      status: { enum: ["intake", "agents_running", "awaiting_review", "document_ready"] },
      created_at: { bsonType: "date" },
      updated_at: { bsonType: "date" }
    }
  }
});
db.projects.createIndex({ user_id: 1 }, { name: "idx_owner" });
db.projects.createIndex({ status: 1 }, { name: "idx_status" });

```

6.2.3 project_intake

```

// migrations/003_create_project_intake.js
db.createCollection("project_intake", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["project_id", "idea_statement", "initial_flow",
      "problem_statement", "proposed_solution", "created_at"],
    properties: {
      project_id: { bsonType: "objectId" },
      idea_statement: { bsonType: "string" },
      initial_flow: { bsonType: "string" },
      suggested_tech_stack: { bsonType: ["array", "null"] },
      problem_statement: { bsonType: "string" },
      proposed_solution: { bsonType: "string" },
      budget_tier: { enum: ["free_only", "low_budget", "unconstrained", null] },
      created_at: { bsonType: "date" }
    }
  }
});
db.project_intake.createIndex({ project_id: 1 }, { unique: true, name: "uniq_project" });

```

6.2.4 agent_runs

```

// migrations/004_create_agent_runs.js
db.createCollection("agent_runs", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["project_id", "agent_type", "version", "status",
      "input_snapshot", "output", "triggered_by", "created_at"],
    properties: {
      project_id: { bsonType: "objectId" },
      agent_type: { enum: ["validation", "research", "gap_detection", "architecture",
        "resource", "roadmap", "risk", "synthesis"] },
      version: { bsonType: "int", minimum: 1 },
      status: { enum: ["queued", "running", "awaiting_review", "accepted"],

```

```

        "needs_review","failed"] },
        triggered_by:{ enum: ["initial_dag", "refinement"] },
        created_at: { bsonType: "date" }
    }
}
});
db.agent_runs.createIndex(
  { project_id: 1, agent_type: 1, version: -1 }, { name: "idx_project_agent_version" });
db.agent_runs.createIndex({ status: 1 }, { name: "idx_status" });

```

6.2.5 refinement_threads, master_documents, recommended_resources, sources

```

// migrations/005_create_remaining_collections.js
db.createCollection("refinement_threads");
db.refinement_threads.createIndex(
  { project_id: 1, agent_type: 1 }, { name: "idx_project_agent_thread" });

db.createCollection("master_documents");
db.master_documents.createIndex(
  { project_id: 1, version: -1 }, { name: "idx_project_version" });

db.createCollection("recommended_resources");
db.recommended_resources.createIndex({ project_id: 1 }, { name: "idx_project" });
db.recommended_resources.createIndex({ resource_type: 1 }, { name: "idx_type" });

db.createCollection("sources");
db.sources.createIndex({ agent_run_id: 1 }, { name: "idx_agent_run" });

```

6.3 Access Control Policies (Row-Level-Security Equivalent)

MongoDB Atlas does not provide native row-level security comparable to Postgres RLS, so Manthan enforces equivalent per-record ownership guarantees at the application layer, inside the API Gateway's authorization middleware (Section 17), backed by the single-root ownership chain described in Section 6.1.

Policy	Enforcement Point	Rule
Project ownership	Gateway middleware, all /projects/:id* routes	If you don't have the same user_id on the resolved project as the JWT subject, then 403
Intake immutability	document-service, PATCH /projects/:id/intake	The write always creates a new version of a project_intake.
Agent run scoping	agent-session-service	Each agent_runs query is filtered by project_id based on the project(s) the authenticated user owns, not on what he/she provides as a filter.
Refinement scope-lock	ai-core Orchestrator node dispatch	Refinement prompt is required to be server-side and be bound to one agent_id (Section 17), not another agent's tool set.
Document access	document-service, master-document routes	The pdf_url is an object-storage URL with a signing key that will not be moved from the service, and with a time limit.
Cross-collection joins	All aggregation pipelines	Each \$lookup stage has an explicit project_id equality match as its first stage to avoid cross tenant leakage.

7. Local Development & Installation Guide

The polyglot, multi-service system (Section 9) called Manthan consists of a Node.js gateway, four domain microservices, a Python/FastAPI agentic core, and three client surfaces. Local orchestration is a source of truth and the following steps are for both the fully containerized path and the direct-on-host path for active development.

7.1 Prerequisites & System Requirements

Requirement	Minimum Version	Used By
Node.js	20.x LTS	To perform these actions, the gateway must have the appropriate permissions. These actions require appropriate permissions at the gateway level.
npm / pnpm	npm 10.x or pnpm 9.x	JS/TS workspace installs
Python	3.11+	ai-core (FastAPI orchestration layer)
pip / poetry	pip 23+ or poetry 1.7+	ai-core dependency management
Docker & Docker Compose	Docker 24+, Compose v2	There is a local, multi-service orchestration (infra/docker-compose.yml) as well.
MongoDB	7.x (local) / Atlas (cloud)	Primary document store
Redis	7.x (local) / Upstash (cloud)	The queue backend to include cache + BullMQ.
Expo CLI	latest	The Android build of apps/mobile (React Native).
Google Chrome	current stable	Please note: You can find the local loading information for apps/extension in chrome://extensions.

7.2 Step-by-Step Setup & Configuration

1. Clone the repository and move into the workspace root.

```
git clone https://github.com/<org>/Manthan.git
cd Manthan
```

2. Copy the environment template and fill in secrets (Section 7.3).

```
cp .env.example .env
```

3. Install JavaScript/TypeScript workspace dependencies (gateway, services, web app).

```
npm install --workspaces
```

4. Install the Python agentic core's dependencies, ideally inside a virtual environment.

```
cd ai-core
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cd ..
```

5. Start the infrastructure dependencies (MongoDB, Redis) via Docker Compose.

```
docker compose -f infra/docker-compose.yml up -d mongo redis
```

6. Seed the database with the collection validators and indexes from Section 6.2.

```
npm run seed-db # runs scripts/seed-db.ts against MONGO_URI in .env
```

7. Bring up every service in watch/dev mode (see Section 7.4 for the full command matrix).

```
npm run dev # concurrently runs gateway + domain services + web app + ai-core
```

8. (Optional) Start the mobile app in Expo Go for on-device testing.

```
cd apps/mobile && npx expo start
```

9. (Optional) Load the Chrome extension locally: open chrome://extensions, enable Developer Mode, "Load unpacked", select apps/extension/dist after building it.

```
cd apps/extension && npm run build
```

7.3 Environment Variables Reference

Variable	Required	Default	Description / How to Acquire
MONGO_URI	Yes	—	The connection string to connect to your MongoDB database (local docker-compose service or Atlas cluster URI).
REDIS_URL	Yes	—	The URI for the Redis connection that is used by local docker-compose service or Upstash URL. The URI for Redis connection used by local docker-compose service or Upstash URL.
JWT_ACCESS_SECRET	Yes	—	Secret 15-minute access tokens signing
JWT_REFRESH_SECRET	Yes	—	Signing Secret for 7-day Rotating Refresh Token
GOOGLE_OAUTH_CLIENT_ID	No	—	Select APIs & Services from Google Cloud Console, and then select the Credentials tab. On the Google Cloud Console, go to the APIs & Services tab and select Credentials.
LLM_PROVIDER_API_KEY	Yes	—	Key for LLM provider used to back all agents' reasoning calls
LLM_PROVIDER	No	claude	Chooses the provider independent back-end (Section 10 — Technology Stack)
SEARCH_API_KEY	Yes	—	The key powering Agent 2's general fan out of the web search provider.
GITHUB_API_TOKEN	No	unauthenticated (rate-limited)	Increases rate limit for the GitHub API for Agent 5

Variable	Required	Default	Description / How to Acquire
KAGGLE_USERNAME / KAGGLE_KEY	No	—	Create New API Token → Account → Kaggle
YOUTUBE_DATA_API_KEY	No	—	YouTube Data API v3 is accessible from Google Cloud Console. YouTube Data API v3 can be accessed from the Google Cloud Console.
OBJECT_STORAGE_BUCKET	Yes	—	A Cloudflare R2 / S3-compatible PDF rendering bucket.
OBJECT_STORAGE_ACCESS_KEY / SECRET	Yes	—	The credentials for the object storage bucket to be used above. The credentials that you will use for the object storage bucket above.
RATE_LIMIT_WINDOW_MS	No	60000	The size of the token bucket limiter (Section 5.3.2) for the Gateway.
RATE_LIMIT_MAX_REQUESTS	No	120	The maximum amount of buckets to be allocated to window per user/IP
NODE_ENV	No	development	An environment flag that is used to indicate the standard node.
PORT	No	4000 (gateway)	Listen port (default) per service, each service has its own default

7.4 Running the Local Development Suite

Individual services can also be run in isolation when working on a single layer:

Command	What It Starts
<code>npm run dev --workspace=apps/gateway</code>	API Gateway only, on PORT (default - 4000)
<code>npm run dev --workspace=services/intake-service</code>	Intake normalization service
<code>npm run dev --workspace=services/agent-session-service</code>	A service that tracks the progress of agent runs, versions and refinements.
<code>npm run dev --workspace=services/document-service</code>	Assemble documents and export them to PDF with a master document assembly + PDF export trigger service.
<code>npm run dev --workspace=services/dashboard-service</code>	Dashboard aggregation service
<code>uvicorn app.main:app --reload</code>	This module is a single AI Core that is deployed using AgenticAI Core (FastAPI) from ai-core/.
<code>npm run dev --workspace=apps/web</code>	The default app for the React/Vite web app is http://localhost:5173 .
<code>npx expo start</code>	Run from apps/mobile/ a React Native Android app.
<code>docker compose -f infra/docker-compose.yml up</code>	All services (and MongoDB + Redis) are fully containerized.

8. Verification, Testing & Quality Assurance

Quality assurance is performed in four steps: static analysis on each push; automated unit/integration/E2E suites that gate merges; the agent output is checked against a golden dataset, and guardrails are put in place on the orchestration layer to ensure the orchestration layer remains resilient even in the event of failures of external tools or LLM calls mid-run.

8.1 Static Type Analysis & Code Audits

Check	Command	Scope
TypeScript type-check	<code>npm run typecheck --workspaces</code>	The Gateway will also expose data to domain services, web app, mobile app, and extension.
ESLint	<code>npm run lint --workspaces</code>	All TypeScript/JavaScript workspaces
Python static typing	<code>mypy app/</code>	ai-core (run from ai-core/):compatible with the latest version of the AI Core.
Python linting	<code>ruff check app/</code>	ai-core
Dependency audit	<code>npm audit --workspaces && pip-audit</code>	The monorepo is fully implemented and builds on each PR.
OpenAPI schema validation	<code>npm run generate-openapi -- --check</code>	This is a work-in-progress, as it needs to be updated with each new version of the live API. This is a work in progress and will need to be updated with every update to the live API in Section 12.

8.2 Algorithm & Agent-Output Validation

Since the usefulness of an agent is not necessarily dependent upon code correctness, the output of agents is judged independently of traditional unit tests by a set of annotated student ideas with human reviewed expected characteristics.

- With Golden set evaluation, every agent type (Section 4.1–4.8) is tested with a fixed benchmark of ~40–60 idea intakes from previous submissions per domain and is judged using a rubric that is designed by an LLM and periodically reviewed by humans.
- This takes every prompt-template change (ai-core/app/prompts/) and compares it to the same "golden" set of prompt-templates before merging, and the score differences are reported in the PR, meaning that if some prompt-template change that improves Agent 4 actually hurts Agent 6 downstream context, it is identified early.
- Correctness of the refinement-loop — A separate Pytest suite makes assertions about the correctness of the refinement-loop, such as that refining one agent does not change the other agent's agent_runs record, that needs_review is raised only when the score of the downstream-impact of one agent crosses the threshold $\tau = 0.5$ (Section 5.2.2).

- Unit tests for mathematical formula tests: the CPM forward/backward pass (Section 5.2.1), the PERT estimator (Section 5.1.3) and the token-bucket limiter (Section 5.3.2) are compared to hand-computed fixtures for which the expected output is known.
- There is a wealth of knowledge out there. There is so much knowledge available.
- Each backend service is an independently deployable container, the web app is a static SPA that communicates with the same Gateway endpoint as the web app and the Android app and Chrome extension are distributed through their respective stores that communicate with the same Gateway endpoint as the web app (no per-platform backend).

8.3 Operational Guardrails & Edge-Case Handling

Scenario		Guardrail
External (GitHub/Kaggle/Scholar/YouTube) unavailable	API	Affected agent does return a partial result with the gap explicitly marked out in the result, but does not block the DAG (Section 3.2)
LLM provider error / timeout		Orchestrator tries again (with bounded exponential backoff) and agent continually switches to Running / Failed / Running (Section 3.4 lifecycle)
Refinement targets an agent still Running		Gateway returns 409; refinement will only be accepted after gateway has reached AwaitingReview or Accepted.
Student edits Stage 1 intake mid-flow		A full DAG re-triggers, not a single layer refinement - the common context all agents rely on has changed (Section 5.3)
Downstream agent's assumption invalidated by a refinement		Note that student has made an explicit choice on whether to propagate or not (Section 5.2): don't silently re-run, flag needs review
Synthesis requested before all layers accepted		Partial/draft export path marks sections which are not fully done, not as placeholders (Section 6.1)
Prompt-injection via scraped web content		All LLM calls (Section 17) have delimited, non-instruction content blocks from the outside.
Refinement prompt attempts to address another agent's tools		An agent_id was passed as a scope lock to the server-side, cross-agent tool invocation is structurally impossible (Section 6.3)

9. Multi-Platform Deployment & DevOps Pipeline

A sample Dockerfile per backend service, in this case the API Gateway service is shown below (the other domain microservices and the ai-core FastAPI application are similar with a Python base image).

9.1 Static Web Deployment (Vercel)

The React/Vite web app (apps/web) deploys as a static build to Vercel, with the Gateway URL injected as a build-time environment variable.

```
# vercel.json (apps/web)
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "framework": "vite",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }],
  "env": { "VITE_API_BASE_URL": "@manthan_api_base_url" }
}

# Manual deploy
cd apps/web
vercel --prod
```

9.2 Containerized Deployment (Docker / Cloud Run)

Each backend service ships its own Dockerfile; a representative example for the API Gateway is shown below (services/domain microservices and the ai-core FastAPI app follow the same pattern with a Python base image).

```
# apps/gateway/Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
EXPOSE 4000
HEALTHCHECK --interval=30s --timeout=5s CMD wget -qO- http://localhost:4000/healthz || exit 1
CMD ["node", "dist/server.js"]

# infra/docker-compose.yml (excerpt)
services:
  gateway:
    build: ../apps/gateway
    ports: ["4000:4000"]
    env_file: ../.env
    depends_on: [mongo, redis]
  ai-core:
    build: ../ai-core
    ports: ["8000:8000"]
    env_file: ../.env
    depends_on: [mongo, redis]
  mongo:
```

```
image: mongo:7
volumes: ["mongo-data:/data/db"]
redis:
  image: redis:7-alpine
volumes:
  mongo-data:
```

Each container image is deployed independently to Google Cloud Run (Section 18), with staging auto-deploying from every merge to main and production gated behind a manual approval step in GitHub Actions.

```
# .github/workflows/deploy.yml (excerpt)
jobs:
  deploy-staging:
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: google-github-actions/deploy-cloudrun@v2
        with:
          service: manthan-gateway-staging
          image: gcr.io/${{ secrets.GCP_PROJECT }}/gateway:${{ github.sha }}
  deploy-production:
    needs: deploy-staging
    environment: production # requires manual approval in GitHub
    if: startsWith(github.ref, 'refs/tags/v')
    steps:
      - uses: google-github-actions/deploy-cloudrun@v2
        with:
          service: manthan-gateway-prod
          image: gcr.io/${{ secrets.GCP_PROJECT }}/gateway:${{ github.ref_name }}
```

9.3 Native Mobile & Extension Compilation

Android (React Native / Expo)

```
cd apps/mobile
eas build --platform android --profile production
eas submit --platform android # uploads to Google Play Console
```

Chrome Extension (Manifest V3)

```
cd apps/extension
npm run build # outputs dist/ with manifest.json + bundled popup/background
zip -r extension.zip dist/
# Upload extension.zip via the Chrome Web Store Developer Dashboard
```

Note: Android app and Chrome extension are deployed to their respective app stores, but both share the same authentication and call to the same Gateway endpoint as the web app (Section 18) — only one backend deployment to manage no matter how many apps the client surface is — one backend for many apps.

10. Project Changelog, Roadmap & License

10.1 Operational History & Version Commit Log

Version	Phase	Summary
v0.1.0	Foundation	Repository scaffolding (Section 9) and Stage 1 intake schema and normalizer (Section 2) implemented; MongoDB collections and validators (Section 6.2) created
v0.2.0	Orchestration Core	Orchestrator and execution DAG (Section 3.3) was set up; Agent 1 (Validation) and Agent 2 (Research) were implemented and connected to the Gateway.
v0.3.0	Analytical Agents	Agent 3 (Gap Detection), Agent 4 (Architecture), Agent 5 (Resource) implemented and per-agent tool-set isolation (Section 15.2) enforced
v0.4.0	Planning & Risk Agents	Agent 6 (Roadmap) implemented with CPM/PERT scheduling (Section 5.2.1) Agent 7 (Risk & Feasibility)
v0.5.0	Human-in-the-Loop Layer	Layer-scoped refinement (Section 5) was shipped, as well as version history UI (Section 5.2.2) and downstream-impact detection (Section 5.2.2).
v0.6.0	Synthesis & Export	Agent 8 (Documentation Synthesis) and the Mermaid → HTML → PDF rendering pipeline (Section 6.3) finished up.
v0.7.0	Cross-Platform Delivery	Android app (Expo) & Chrome extension (Manifest V3) were launched, with a common design system and API client as the web app (Section 16)
v0.8.0	Hardening	All the cases have been rate limited, JWT rotated, prompt-injection mitigated (Section 17) and load tested (k6, Section 20).
v1.0.0	Hackathon Release Submission	Enhanced with virtually all the features from intake, all 8 agents, refinement, multi-platform delivery, and PDF export; full test suite green

10.2 Future Roadmap & Milestone Objectives

- Add further layers of "per-team" collaborative editing to the layer-level ability to do edits as described in Section 5 with a conflict-aware merge.
- Addition of pluggable document layers — an additional layer of agents in the synthesis, which are a new node to the existing DAG (Section 3.3), but that do not require any modification to the existing agent.
- Once the number of documents exceeds the current clustering threshold that can be achieved by using a prompt with an LLM, a fine-tuned variant or retrieval-augmented version of Agent 3 is used per domain (health-tech, agritech, civic-tech).
- Revisit "no vector database at this scale" decision (Section 15.3) once a fairly large number of sources (few hundred) in a project's research corpus regularly cross the threshold.
- Multi-region deployment — deploy stateless container fleet to a second Cloud Run region, behind the CDN, as described in the multi-region readiness described in Section 19.
- Surface for viewing a student's agent board and Master Document with the ability to add comments, but not edit the document, provided by faculty/mentor.

10.3 Troubleshooting Matrix

Issue	Root Cause	Resolution
Agent stuck in Running indefinitely	The External API (GitHub/Kaggle/Scholar) is not showing up in the Orchestrator timeout.	Make sure that the rate-limit throttling config (Section 19) is correct per provider, and that the bounded-retry backoff (Section 3.2) is set with a finite max-attempts.
Refinement returns 409	Target agent isn't yet AwaitingReview / Accepted	Do a poll of /agents/:agentType with status of Running in order to submit a refinement for it (Section 12).
Downstream agent unexpectedly flagged needs_review	There was a change in the field and a crossing of $\tau = 0.5$ for impact(d) score (Section 5.2.2)	What it should do — check the flagged diff and decide if the diff is worth propagating as a bug, or just leave it in the development tree
PDF export missing a section	The following layer(s) were requested and Accepted with allow_partial=true:	Confirm intended if not, wait for all 7 analytical agents to be Accepted/default-accepted (Section 6.1).
Local docker compose fails to reach MongoDB	The .env file will include MONGO_URI which is pointing to atlas, instead of the local mongo service.	If operating the containerized stack (Section 7.2 - 7.3) set MONGO_URI=mongodb://mongo:27017/manthan
Chrome extension shows "not authenticated"	The extension could not be refreshed because the service worker session token it received expired.	Open the popup to work around the refresh-token exchange; check secure device storage (Section 17)
Golden-set evaluation score regresses after a prompt change	The output schema conformance for an agent was changed by changing the prompt template	Compare the prompt with version history of the ai-core/app/prompts directory, and run the golden-set suite (Section 8.2) prior to merging
Rate limit triggers under normal single-user load	The RATE_LIMIT_MAX REQUESTS was set to an extremely small value, which is only suitable for a legitimate WebSocket-based polling.	Adjust the capacity parameter B of the tuner (Section 5.3.2 / 7.3 env reference), instead of turning the limiter off.

10.4 Error Log Book

A representative catalog of substantive technical hurdles encountered during development, their root causes, and the resolution strategy adopted — kept as an ongoing appendix rather than a one-time record.

Hurdle	Root Cause	Resolution Strategy
Refinements occasionally mutated unrelated agents' state	The first iteration of Orchestrator was to run the entire DAG subgraph again, rather than just a conditional-edge re-entry, if it was refined.	The LangGraph state graph (Section 15.1) was redesigned to be a single-node conditional re-entry, to avoid re-executing unwanted paths in the loop.
Gap Detection Agent produced inconsistent clusters across near-identical corpora	Order-sensitivity with respect to the input source list was observed for naive prompt-based clustering.	Conducted deterministic pre-sorting of research corpus by canonical URL prior to clustering, and a fixed-seed instruction in the clustering prompt.
PDF rendering blocked agent-reasoning workers under load	Initially, all Headless-browser rendering jobs (CPU-bound) used the	Separate PDF rendering in a separate worker pool (Section 19), which allows to

Hurdle	Root Cause	Resolution Strategy
	same pool of workers as all LLM-reasoning jobs (I/O-bound).	separate the CPU intensive and I/O intensive workloads.
Downstream agents silently went stale after a refinement	The first few ones automatically propagated changes upstream without marking them.	Maintained explicit Impact(d) scoring and needs review flag (Section 5.2.2) and did not replace silent propagation (Section 5.2.1) which was left in control of the student;
Duplicate citations appeared across the bibliography appendix	Each agent independently deduplicated sources without using other agents' sources	Improved the citation deduplication pass of the Synthesis Agent (Section 4.8) by using the canonicalized URL as the key (Section 4.9).
Extension session state diverged from web/mobile	The agent status was initially stored locally, rather than loaded live from the Gateway, by extension.	Added the WebSocket/REST contract to both Gateway and Extension, and standardized all three surfaces (Section 16.5)

10.5 Licensing & Intellectual Property Terms

Manthan is released under the MIT License.

MIT License

Copyright (c) 2026 Manthan Project Contributors

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.